

Global Translation Platform

Demo Architecture Overview

Purpose

This document explains the logic behind the demo: how the quote is calculated, the proposed technology stack, and how Shopify Plus and the Laravel/Filament application work together.

1. Calculator Logic

The quote engine is driven by structured pricing data rather than hard-coded product prices. The customer selects the destination market, language pair, document details and delivery tier; the server-side pricing engine then returns Standard, Priority and Express options.

Input	Example
Market	United States
Language pair	Spanish -> English
Certification	Certified translation (USCIS)
Pricing unit	Page
Quantity	4 pages
Base retail rate	\$35 / page
Minimum	\$35
Priority multiplier	+30%
Express multiplier	+70%
Options	Apostille / shipping / copies if selected

Illustrative formula

`price = max(minimum, units x rate[language pair, certification, market]) x delivery tier + options`

Each quote should store the pricing-rule version used at the time it was created. That keeps historical quotes auditable even if pricing changes later.

2. Proposed Tech Stack

Layer	Technology / responsibility
Customer storefront	Shopify Plus theme / storefront experience
Commerce	Shopify Markets, checkout, payments, customers and orders

Custom application	Laravel backend for quote logic, pricing, quote records, documents, automation and operations
Admin / operations	Filament for quotes, orders, translator management, pricing administration and workflow
Data	Application database for quotes, pricing versions, translators, assignments, documents and orders
Secure files	Private object storage with signed / expiring links
Automation	Laravel jobs / scheduler plus email and WhatsApp integrations

3. How Shopify and Laravel / Filament Work Together

The key design principle is simple: Shopify handles commerce; Laravel handles translation-specific business logic and operations.

Customer enters quote details -> Laravel calculates and saves quote -> customer accepts quote -> Shopify Draft Order / checkout handoff is created -> customer pays in Shopify -> Shopify payment/order webhook updates Laravel -> the paid order appears in Filament and enters the operations workflow.

Before payment: Laravel is the system of record for the quote, uploaded documents, selected service, pricing version, attribution and recovery status.

At checkout: Laravel validates the quote server-side and creates the Shopify handoff. Shopify becomes responsible for payment and the commerce order.

After payment: Shopify notifies Laravel through webhooks. Laravel creates or updates the operational job, and staff manage translator assignment, production status, QA and delivery from Filament.

This split avoids recreating checkout and payments in Laravel while keeping the custom translation workflow outside Shopify's product/order model.

4. Demo Contents

Front-end demo: a clickable six-step quote experience covering destination, languages, document upload, Standard / Priority / Express pricing, lead capture and checkout handoff.

Filament-style admin demo: an operational view showing recent quotes/orders, translator directory/profile and a simple pipeline view.

The demo is intentionally visual only. It does not upload real files, call Shopify APIs, calculate live prices, create orders or connect to a backend.